

Modelos CapsNets

CPE883 - Tópicos Especiais em Machine Learning

Vivian Rodrigues (DRE:125228569)

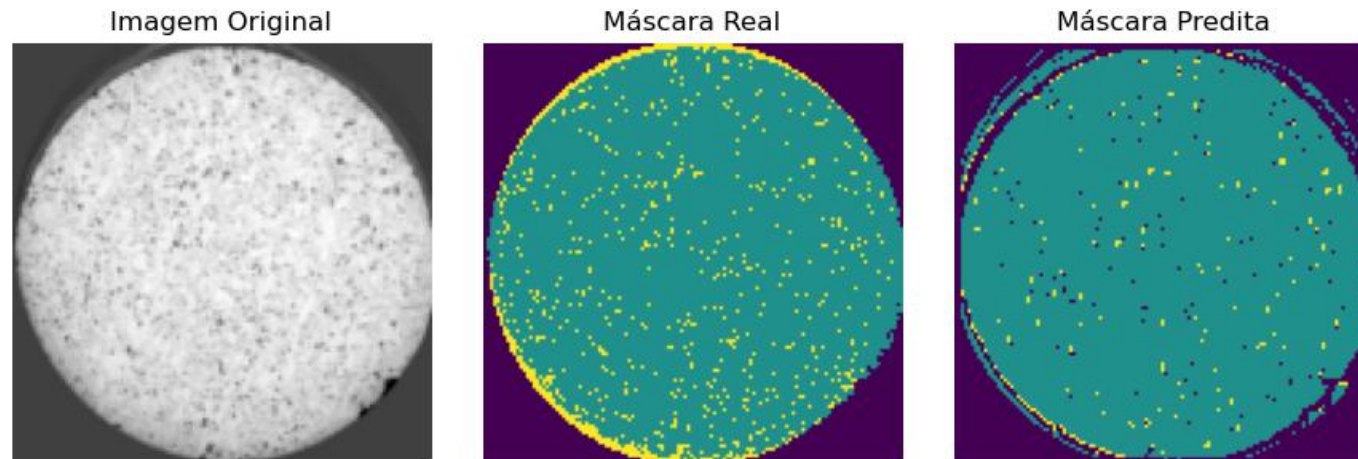
Introdução e Objetivo

Objetivo:

- Avaliar desempenho preliminar de CapsNet na segmentação de imagens de microtomografia de rochas.

Saída esperada:

- Máscara segmentada com 3 classes (0=fundo, 1=poro, 2=rocha).



Divisão do dataset: Treino, validação e teste

Objetivo:

Separar o dataset original em três conjuntos para garantir, treinamento eficaz, avaliação justa e evitar overfitting

Função `split_dataset(...)`
`random_split(dataset, [train_size, val_size, test_size],
generator=torch.Generator().manual_seed(seed))`

- Treino (70%): usado para ajustar os pesos do modelo
- Validação (15%): monitora desempenho durante o treino (tuning)
- Teste (15%): avalia generalização após o treinamento

Semente aleatória (seed) para garantir reprodutibilidade
Garante que a soma das proporções é 100%

```
#Split treino, teste e validação
def split_dataset(dataset, train_ratio=0.7, val_ratio=0.15, test_ratio=0.15, seed=42):
    assert abs(train_ratio + val_ratio + test_ratio - 1.0) < 1e-6, "Soma das proporções deve ser 1"
    total_size = len(dataset)
    train_size = int(train_ratio * total_size)
    val_size = int(val_ratio * total_size)
    test_size = total_size - train_size - val_size
    return random_split(dataset, [train_size, val_size, test_size],
generator=torch.Generator().manual_seed(seed))
```

Arquitetura Detalhada

Entrada: Imagens de tamanho 128×128, com 1 canal (grayscale)

Camadas:

1) Conv2d

→ Extrai features iniciais da imagem

→ Saída: [B, 32, 128, 128]

2) PrimaryCaps

→ Projeta os features em cápsulas vetoriais (grupos de neurônios)

→ Saída: [B, caps_channels=8, caps_dim=8, H, W]

3) Squash (ativação)

→ Normaliza os vetores das cápsulas

→ Mantém direção (informação) e comprime a magnitude para [0, 1] (confiança)

4) Reorganização (view)

→ Achata a dimensão vetorial: [B, 64, H, W]

→ Prepara para convolução tradicional

```
class SegCapsNet(nn.Module):
    def __init__(self, img_channels=1, num_classes=3):
        super().__init__()
        self.conv1 = nn.Conv2d(img_channels, 32, kernel_size=5, stride=1, padding=2)
        self.primary_caps = PrimaryCaps(32, caps_channels=8, caps_dim=8,
        kernel_size=1, stride=1)
        # stride=2 removido para não dobrar tamanho
        self.deconv = nn.ConvTranspose2d(8*8, 32, kernel_size=1, stride=1)
        self.final = nn.Conv2d(32, num_classes, kernel_size=1)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        caps = self.primary_caps(x)
        caps = caps.view(x.size(0), -1, caps.size(3), caps.size(4)) # [B, C*dim, H, W]
        x = F.relu(self.deconv(caps))
        out = self.final(x)
        return out
```

Arquitetura Detalhada

5) ConvTranspose2d

→ Refina os mapas e mantém resolução (sem upsampling neste caso)

→ Saída: [B, 32, 128, 128]

6) Conv2d final

→ Gera os logits de segmentação

→ Saída: [B, 3, 128, 128] (3 classes por pixel)

Saída final:

Tensor de logits [B, 3, 128, 128] → Um mapa por classe, para cada pixel.

A função squash:

- vetores com pequena norma (fraca evidência da entidade) são "encolhidos" para perto de zero em
- Vetores os da grande norma (forte evidência) se aproximem de norma 1.

Ela garante que a informação contida no vetor é representada pela direção, enquanto o comprimento indica a probabilidade de existência da entidade detectada.

```
class PrimaryCaps(nn.Module):
```

```
    def __init__(self, in_channels, caps_channels, caps_dim, kernel_size, stride):
        super().__init__()
        self.caps_dim = caps_dim
        self.caps_channels = caps_channels
        self.conv = nn.Conv2d(in_channels, caps_channels * caps_dim, kernel_size,
                               stride)
```

```
    def forward(self, x):
```

```
        batch = x.size(0)
```

```
        out = self.conv(x)
```

```
        out = out.view(batch, self.caps_channels, self.caps_dim, out.size(2), out.size(3))
```

```
        out = self.squash(out)
```

```
        return out
```

```
    def squash(self, s):
```

```
        norm = torch.norm(s, dim=2, keepdim=True)
```

```
        return (norm**2 / (1 + norm**2)) * (s / (norm + 1e-8))
```

Função de Custo, Otimização e métricas

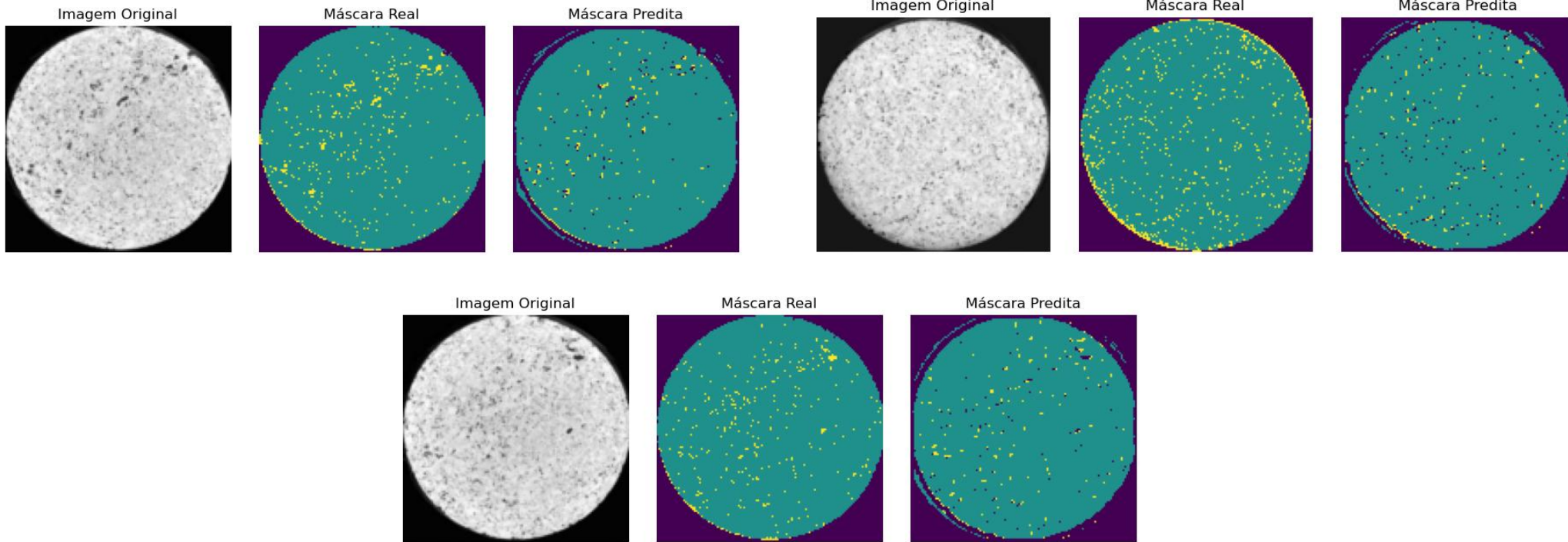
- Função de custo: CrossEntropyLoss (multi-classe)
- Otimização: Adam(lr=1e-3)
- Batch size: 4
- Épocas: 3 (teste preliminar)
- Métricas: Dice Score, IoU

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|} \quad \text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

- Resolução entrada/saída: 128x128

```
# ==== Métricas ====  
def dice_score(pred, target, num_classes):  
    dice = []  
    pred = pred.cpu().numpy()  
    target = target.cpu().numpy()  
    for c in range(num_classes):  
        pred_c = (pred == c).astype(np.uint8)  
        target_c = (target == c).astype(np.uint8)  
        inter = (pred_c * target_c).sum()  
        union = pred_c.sum() + target_c.sum()  
        if union > 0:  
            dice.append(2.0 * inter / union)  
    return np.mean(dice)  
  
def iou_score(pred, target, num_classes):  
    ious = []  
    pred = pred.cpu().numpy()  
    target = target.cpu().numpy()  
    for c in range(num_classes):  
        pred_c = (pred == c).astype(np.uint8)  
        target_c = (target == c).astype(np.uint8)  
        inter = (pred_c * target_c).sum()  
        union = pred_c.sum() + target_c.sum() - inter  
        if union > 0:  
            ious.append(inter / union)  
    return np.mean(ious)
```

Resultados Preliminares (600 imagens)



Epoch	Dice	IoU
1	0.46	0.37
2	0.72	0.64
3	0.77	0.69

Conclusões e próximos Passos

Conclusões:

- CapsNet segmenta com boa precisão em poucas épocas
- Estrutura de cápsulas preservam as informações espaciais
- As métricas mostram aumento rápido de acurácia

Próximos passos:

- Implementar shuffle no val_loader (amostras aleatórias) – a confirmar
- Incluir rastreabilidade por amostra
- Evoluir com CKAN e Diffusion (U-net)
- Rodar com todas as imagens
- Comparar resultados com CKAN e Diffusion Models